

RESEARCH ARTICLE

Leveraging Key Source Code Metrics for Predicting Software Vulnerabilities With Machine Learning

BEHZAD KHALAJI EMAMZADEH ABBASI AND ALIREZA SHAMELI-SENDI[✉]

Faculty of Computer Science and Engineering, Shahid Beheshti University (SBU), Tehran 1983969411, Iran

Corresponding author: Alireza Shameli-Sendi (a_shameli@sbu.ac.ir)

ABSTRACT Security vulnerabilities are a common concern in software systems, posing significant risks such as unauthorized access, data breaches, and system failures, which makes their early detection essential for both organizations and end-users. To address this challenge, various techniques—particularly machine learning—have been explored. This study investigates the key source code factors that contribute to software vulnerability prediction using multiple machine learning models, with the dual objective of detecting vulnerability presence and classifying vulnerability types such as Denial of Service and Buffer Overflow. Unlike prior studies that largely overlook the relationship between code metrics and specific vulnerability categories, this work systematically analyzes these connections to provide actionable security insights. The proposed approach is evaluated on datasets from ten real-world projects, including OpenSSL, SQLite, FreeType, LibTIFF, Libxslt, Binutils, FFmpeg, ImageMagick, OpenSC, and rdesktop, while addressing class imbalance through the SmoteTomek technique. Experimental results demonstrate that Random Forest achieves superior performance, reaching an accuracy of 0.96, precision of 0.948, recall of 0.974, and F1-score of 0.961 in vulnerability detection, along with strong effectiveness in vulnerability type classification. Overall, the findings highlight the most influential source code metrics in vulnerability prediction and confirm the practical value of machine learning for enhancing secure software development.

INDEX TERMS Source code metrics, vulnerability, code property graph, machine learning.

I. INTRODUCTION

Software security has become a critical concern due to the rapid increase in cyberattacks on both private organizations and public infrastructure. Among the many threats, software vulnerabilities occupy a central role. They are often subtle flaws that may seem harmless at first. However, once exploited, such weaknesses can compromise entire systems. Because of their serious consequences, including data breaches and service disruptions, preventing and mitigating vulnerabilities has long been a priority for researchers [1], [2], [3]. The risks are evident in real-world incidents. The WannaCry ransomware, for example, forced hospitals, communication providers, and transportation systems to shut down [54]. This single attack caused financial losses

amounting to hundreds of millions of dollars. Other vulnerabilities enable privilege escalation, denial-of-service (DoS) attacks, or even complete system compromise. The Heartbleed bug illustrates the severity of this issue. Just a few lines of faulty code exposed billions of Internet users to critical security risks.

Traditional approaches to software security rely heavily on manual code inspection. While effective to some extent, this method is highly time-consuming and requires expert knowledge. It is also prone to overlooking issues that stem from subtle programming conventions or from changes in evolving codebases [11], [12], [13]. Many vulnerabilities remain hidden because they do not immediately affect normal functionality. As a result, attackers may exploit them long before detection. With the increasing scale and complexity of modern software, manual inspections alone are no longer sufficient. This challenge has created a strong demand for

The associate editor coordinating the review of this manuscript and approving it for publication was Mahdi Zareei[✉].

semi-automatic and fully automated techniques [2], [13], [15], [16], [17], [18], [19], [20], [21].

In recent years, numerous studies have delved into the realm of software vulnerability detection, with the approaches proposed in these studies broadly falling into the following categories: (1) *Static Analysis*: This category involves examining the source code or the compiled binaries without actually executing them. Such techniques are proficient in uncovering vulnerabilities such as buffer overflows, integer overflows, and various coding errors. Notable examples of static analysis tools include Coverity [22], CodeSonar [23], [24], and FindBugs [25], (2) *Dynamic Analysis*: Dynamic analysis focuses on scrutinizing the behavior of software during its execution. These techniques are adept at identifying vulnerabilities like injection attacks, cross-site scripting (XSS) attacks, and other vulnerabilities that only manifest during runtime. Prominent dynamic analysis tools include AppScan [26], Burp Suite [27], and Fuzz Testing [28]. (3) *Hybrid Approaches*: Hybrid approaches amalgamate both static and dynamic analysis techniques to augment the precision of vulnerability detection. These methods are capable of identifying both known and hitherto unknown vulnerabilities [29].

Furthermore, in contemporary times, the utilization of machine learning techniques for the prognosis and diagnosis of software vulnerabilities has become increasingly prevalent, with numerous studies conducted in this domain [4], [14], [18], [20], [21], [30], [31], [32]. To forecast software vulnerabilities through machine learning, it is imperative to amass relevant data pertaining to the software under scrutiny. This dataset may encompass particulars such as the software's source code, entry and exit points, system calls, and more. Subsequently, a diverse array of machine learning algorithms is employed to pinpoint the software's weak points and vulnerabilities.

Recent years have seen substantial research activity on the application of machine learning and deep learning methods for predicting software vulnerabilities. Current best, of, breed methods have shown the ability to deliver good results by using handcrafted features, learned representations, or deep neural architectures. On the other hand, the majority of these works deal only with binary vulnerability prediction, i. e., classifying a software component as vulnerable or not, and they do not thoroughly analyze the relationship between source code features and vulnerability types.

Besides, a good number of recent studies depend on sophisticated feature representations or black, box models that usually lack interpretability and hardly provide any useful insights for developers. Moreover, the issue of class imbalance, which is inherent to real, world vulnerability and has a significant impact on the performance of vulnerability prediction models, is not always consistently or thoroughly addressed in the prior work.

Advancing the state of the art means this study deals with the above, mentioned weaknesses through an overall and explainable machine learning framework. To this end, first,

our work does not stop at binary vulnerability detection but it can jointly predict the presence of vulnerabilities and their types. Secondly, we use multiple feature selection techniques to systematically investigate the contribution of source code metrics to different categories of vulnerabilities, which results in a better understanding of which code, level features are the main drivers of each type of vulnerability. Next, class imbalance is one of the issues that we directly handle by using the SMOTE-Tomek technique and, after that, we check its effect on ten software products from the real world by applying strong cross, validation methods. Combining interpretable source code metrics, balanced learning, and ensemble, based classifiers, this work not only gives better predictive performance but also delivers practical, code, level insights, thus making a two-way contribution to the development of state, of, the, art software vulnerability prediction methods.

In the context of our research, we initially apply various methodologies for feature selection in machine learning to the existing dataset. The objective is to identify the most critical factors in predicting software vulnerabilities, with the ultimate aim of assisting software developers in crafting more secure code. After the identification of these pivotal features in vulnerability prediction, we address issues within the dataset, including class imbalances. Subsequently, we implement several well-established machine learning algorithms to predict vulnerabilities and classify the type of attack. In comparison to preceding studies, the model proposed in this research not only demonstrates significantly improved accuracy in detecting vulnerable functions within the source code but also exhibits exceptional precision in identifying the corresponding vulnerability type associated with these functions.

The major contribution of this research are as follows:

- Our investigation delves into the significance of features for predicting various types of vulnerabilities, including but not limited to *DoS*, *Buffer Overflow*, *Memory Corruption*, *SQL Injection*, *Code Execution*, *Bypass* (ASLR Protection, Config and Sanity Check), *Gain Sensitive Information*, and *Gain Privilege*. Notably, most existing studies primarily focus on binary vulnerability prediction and do not explicitly examine the relationship between source code metrics and specific vulnerability types. Recent state-of-the-art works, such as those presented in [11], [14], [20], [21], and [31], mainly address the detection of vulnerability presence using machine learning or deep learning techniques, while the systematic analysis and classification of multiple vulnerability categories remain largely unexplored. In contrast, this study explicitly models and analyzes different vulnerability types, thereby extending the scope of previous research efforts.
- To address the issue of class imbalance between vulnerable and non-vulnerable samples in predicting vulnerability presence, we employ the *SmoteTomek* approach. It is worth highlighting that by using the

SmoteTomek approach, the predictive accuracy of vulnerability presence has improved by 13.6 percent compared to not using it. The results of this paper demonstrate an improvement of approximately 8 percent in accuracy compared to the previous best-performing study in vulnerability presence detection, as reported in [42].

- Additionally, in the context of detecting vulnerability types, we also apply the *SmoteTomek* approach to tackle the imbalance challenge between vulnerable and non-vulnerable samples. This strategic implementation leads to enhanced detection results. Specifically, by using the SMOTE approach, the accuracy of detecting various vulnerability types, as measured by the Accuracy criterion, improves as follows: 9.8 for DoS, 10 for Bypass, 4.4 for Code Execution, 3.2 for Memory Corruption, 2.9 for Buffer Overflow, 2.3 for Privilege Gain, 14.7 for Gain of Sensitive Information, and 1.9 for SQL Injection. The results of this study have shown a 9 percent improvement compared to the previous best work in the field of identifying the types of vulnerability, based on the accuracy metric.

The remaining sections of this paper are organized as follows: Section II provides an overview of related work in the field of source code vulnerability prediction. In Section III, we delve into the details of the proposed methodology. Section IV offers a comprehensive analysis of the dataset employed and presents the key findings of this research. Finally, in Section V, we draw conclusions based on our findings and provide valuable suggestions for potential avenues of future research.

II. RELATED WORK

In previous research, various approaches have been explored to predict software vulnerabilities, each with its own strengths and limitations. Here, we provide an overview of some of these approaches:

Shin and Williams [33] used code complexity as a predictor for vulnerabilities in JavaScript projects. While they achieved a low false positive rate, their approach had a relatively high false negative rate, indicating a need for improvement. In [49], authors employed a combination of code complexity, code reduction, and development metrics to predict vulnerabilities. Using linear diagnostic analysis and Bayesian networks, they achieved an 80 percent recovery rate but had a 52 percent false positive rate. Some researchers [50] attempted to adapt error detection methods to predict software vulnerabilities. They applied decision trees, random forest, logistic regression, and simple Bayes models to Mozilla Firefox versions. These models achieved an accuracy of approximately 70 percent.

Zimmermann et al. [16] expanded the list of metrics considering code leakage, code complexity, code coverage, organizational measures, and real dependencies. They used logistic regression to predict vulnerabilities in Windows Vista-specific code and achieved 60 percent accuracy.

Yu et al. [11] explored various features, including software metrics, the number of subclasses, and the number of functions in a file. They predicted vulnerabilities at the file level, reducing the manual code checking workload for human experts. Zhou and Sharma [51] used a K-fold stacking algorithm to analyze quantity messages for vulnerability prediction. However, it is worth noting that in [14], it was found that both humans and machine learning algorithms struggled to predict failures or errors solely based on quantity messages.

Numerous studies have investigated software vulnerability prediction using machine learning [35], [36], [37], [38]. Li et al. [39] introduced VulDeeLocator, a vulnerability detection system employing deep neural networks and natural language processing to achieve high detection and localization capabilities. Chakraborty et al. [31] scrutinized the pros and cons of using deep neural networks in software vulnerability detection, exploring questions about the research community's understanding of this field. Li et al. [18] employed multi-class support vector machine (SVM) to discern different vulnerability classes, while Perl et al. [40] used commit messages for program vulnerability identification, employing SVM models. Li et al. [20] presented a static analysis system employing deep neural networks to identify software code vulnerabilities, along with the associated features necessary for vulnerability identification. Chen et al. [41] introduced a systematic framework for detecting vulnerabilities in C/C++ source code, leveraging bidirectional recurrent neural networks (RNN). In [32], researchers investigated a novel method for detecting software vulnerabilities at the statement level using neural networks. In recent years, the application of large language models (LLMs) in software vulnerability detection has become increasingly widespread due to their ability to capture complex syntactic and semantic relationships within code. Mechri et al. [55] proposed SecureQwen, a novel vulnerability detection tool that leverages a decoder-only transformer-based LLM with an extended context length of 64K tokens to analyze large-scale Python codebases. Trained on over 1.875 million function-level Python code snippets collected from diverse sources, including GitHub, the Code-parrot dataset, and synthetic LLM-generated data, SecureQwen accurately classifies vulnerabilities across 14 common weakness enumerations (CWEs) such as OS Command Injection, SQL Injection, and Path Traversal. Experimental results demonstrate high detection accuracy with F1 scores ranging from 84% to 99%, showing effective identification of vulnerabilities in both human-written and AI-generated code.

In broader cybersecurity domains, significant research efforts have been made to develop innovative methods for detecting and mitigating cyber threats. Hussain proposed a comprehensive AI-driven cybersecurity framework that integrates Reinforcement Learning with Ensemble Methods to effectively identify and counter dynamic adversarial attacks [56]. Although this study does not directly address software vulnerability detection, it demonstrates the potential

of combining adaptive learning algorithms and ensemble techniques to enhance system resilience against evolving threats. Validated on up-to-date IoT datasets, the framework achieved high detection accuracy and low false positive rates, indicating promising directions for future advancements in automated vulnerability management and secure system design.

In contrast, the proposed approach in this research considers different metrics extracted from the source code and employs machine learning algorithms to establish a relationship between these metrics and the type of vulnerability. This focus on identifying the correlation between source code metrics and vulnerability types distinguishes this research from previous works. By uncovering these relationships, the study aims to enhance vulnerability prediction accuracy and contribute to the field of software security.

III. PROPOSED MODEL

The proposed approach's architecture in this research is depicted in Figure 1. Broadly, this architecture comprises two phases: the *model construction* phase and the *diagnosis* phase. In the construction phase, the model takes as input a section of code from a software program. Initially, the source code is transformed into a graph using a fuzzy analyzer. This graph amalgamates three essential graphs, maintained by the graph database: (1) Abstract syntax tree, (2) Control flow graph, and (3) Program dependency graph.

Once the graph is generated, a set of source code metrics is computed for each function, determined by traversing the graph. Subsequently, the primary dataset is constructed. Each record in this dataset corresponds to a function, and the dataset features encompass source code metrics, function vulnerability outcomes, and vulnerability types. Information regarding vulnerability types is inferred from NVD and CVE databases.

Finally, machine learning models are implemented to discern the vital metrics for diagnosing the presence and type of vulnerability. It is noteworthy that the output of the model construction phase comprises the most effective machine learning models, in conjunction with the key metrics for vulnerability and vulnerability type diagnosis. Both outputs from the model construction phase serve as inputs for the diagnosis phase.

In the diagnosis phase, a graph is initially generated, followed by the calculation of source code metrics from the graph, based on the critical metrics derived from the model construction phase. Ultimately, machine learning models are applied to identify vulnerable functions and ascertain their specific vulnerability types.

A. CODE PROPERTY GRAPH

In this research, a top-down fuzzy parser is employed, alongside a multi-stage parse tree, to generate an abstract syntax tree from the source code. Abstract syntax trees are typically generated by parsers, which come in two main types: *top-down* and *bottom-up* [15]. In this study, we utilize

a top-down fuzzy parser to convert the source code into a tree-like structure. This parser operates in multiple stages. Furthermore, we make use of *Grammer Island* as a tenseless language command for the proposal parser. Grammer Island is a command that specifically describes a portion of the target language and is primarily used in situations where a full syntactic analysis of the input is not required [25].

As mentioned earlier, an abstract syntax tree is generated, and changes in the tree structure are introduced through the fuzzy analyzer. Unlike a parse tree, the abstract syntax tree omits some of the structural details of the program that do not affect the program's meaning. For example, an abstract syntax tree typically excludes syntax symbols such as semicolons, brackets, or parentheses because these elements are implicitly represented within the tree structure. However, the abstract syntax tree does not readily facilitate the examination and control of the execution order of program commands, which is a crucial requirement for modeling, scanning, and discovering program vulnerabilities [6], [35], [45]. To address this need, the program's control flow graph is created, which comprises nodes representing controlled or uncontrolled statements, as well as input and output nodes.

Control flow graphs depict the primary control flow relationships within a program. Generating such a graph is a prerequisite for constructing more specialized representations like program dependency graphs. The program dependency graph serves to illustrate the flow relationships within a software program [35], [45]. In other words, the utilization of a data dependency graph is vital for identifying program vulnerabilities. For instance, vulnerabilities stemming from incorrect value assignments to shared variables can result in attacks on a program's memory. Such vulnerabilities can be effectively assessed by examining how program data changes occur.

To generate code feature graphs, we employ the *Joern* tool [55], which encompasses several core components within its architecture. One of these components is a fuzzy parser that transforms the source code into a parse tree using predefined grammar rules. To generate a custom parser, we employ the *ANTLR* tool as a top-down parser. It is important to note that the grammar used in the *Joern* tool is limited to extracting parse trees for software programs written in C or C++ programming languages. As a result, this portion of the research, which focuses on identifying malware, is constrained to evaluating software projects developed using these programming languages. Subsequently, the other components of the *Joern* tool, derived from the parse tree, create the abstract syntax tree, control flow graph, and program dependency graph.

B. SOURCE CODE METRICS

The dataset employed in this study encompasses 14 metrics derived from the source code, as detailed in Table 1. The source code metrics in Table 1 are not entirely novel and some of them have been explored individually in the contexts of software engineering and vulnerability prediction. However,

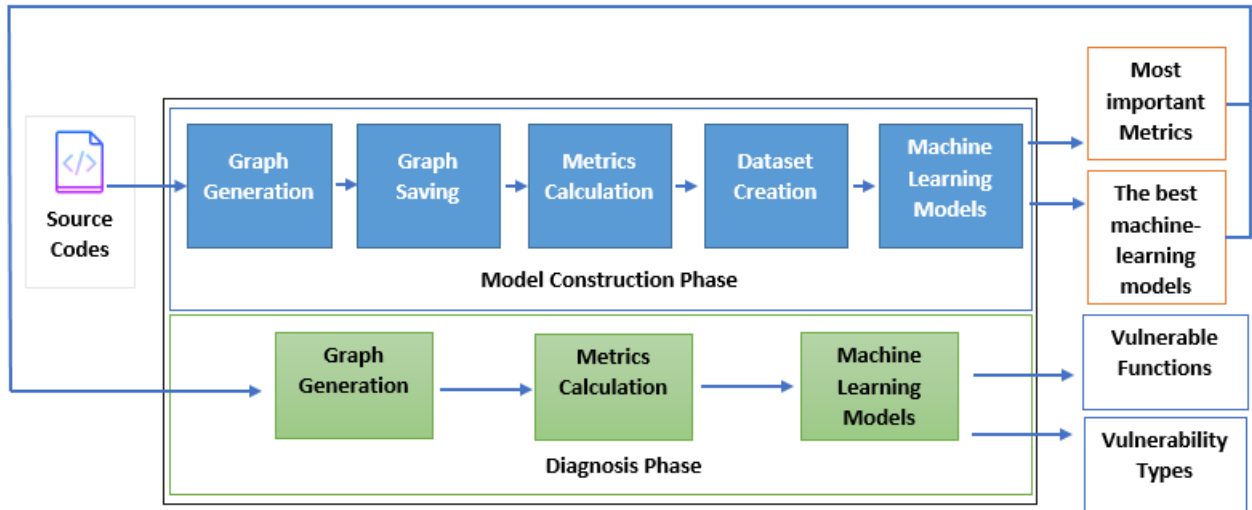


FIGURE 1. The architecture of the proposed approach for identifying the relationships between source code metrics and the type of vulnerability using machine learning.

TABLE 1. Source code metrics.

Description of metric	ID
Cyclomatic complexity	M1
The number of loops in the function from the source code	M2
The maximum level of nested loop in the function from the source code	M3
Number of variables in the function parameter (function argument)	M4
Number of variables prepared in the caller function argument for the recalled function	M5
Number of arithmetic pointers	M6
Number of variables related to arithmetic pointers	M7
Maximum number of arithmetic pointers in relation to a variable	M8
Number of nested control structure terms	M9
The highest level in nested control structure terms	M10
Maximum control dependency between control structures	M11
Maximum data-dependency between control structures	M12
Number of if structures without else	M13
Number of variables involved in conditional-control terms	M14

the majority of the current state, of, the, art methods tend to use these metrics separately, or they rely on a small subset of these, or they apply them only for the task of binary vulnerability detection. This paper, on the other hand, combines a full set of source code metrics and thoroughly investigates their significance for both vulnerability presence detection and multi-class vulnerability type classification. Moreover, while many recent methods depend on complicated or learned feature representations, the metrics in this paper are human, interpretable, so the contribution of different code characteristics to various vulnerability categories can be more clearly understood. As depicted in Table 1, these 14 metrics, extracted from the source code, may exert varying impacts on the prediction of vulnerabilities and the detection of vulnerability types. Consequently, establishing the relationships among these metrics in predicting vulnerabilities and identifying vulnerability types holds significant importance. Figure 2 illustrates which section of the Code Property Graph is utilized for the calculation of each metric. For instance,

to compute the Cyclomatic Complexity metric, only the Control Flow Graph section is employed. However, when determining the metric for the number of variables related to arithmetic pointers, it is essential to utilize both the Abstract Syntax Tree and the Program Dependence Graph sections.

In summary, it can be deduced that the Abstract Syntax Tree plays a pivotal role in the computation of all metrics.

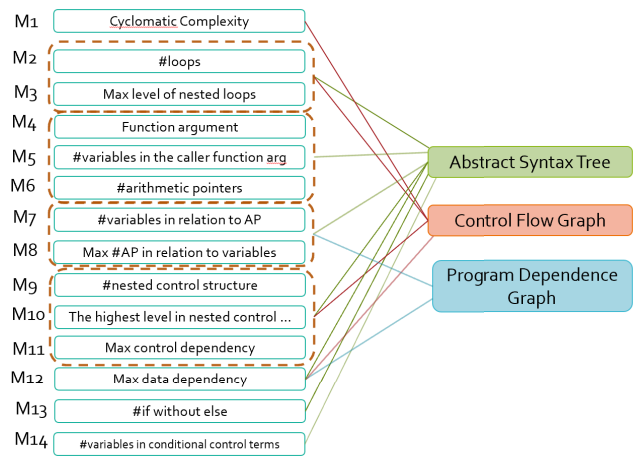


FIGURE 2. The relationship between the calculation of code metrics and sub-sections of the code property graph.

In this investigation, as illustrated in Figure 1, an initial step involves subjecting the dataset to various feature selection methods to pinpoint the most important and optimal features and metrics for predicting software vulnerabilities. Feature selection is a potent technique for cherry-picking subsets of relevant features, offering a range of advantages, two of which are particularly noteworthy: (1) *Enhancing Machine Learning Model Performance*: By curtailing the number of irrelevant [13], redundant [14], or noisy features, feature

selection can boost the accuracy, speed, and simplicity of a model, leading to superior performance on test data and (2) *Mitigating Overfitting*: Feature selection can aid in preventing overfitting by paring down model complexity and reducing the risk associated with irrelevant or redundant features that might not generalize effectively to new data.

Given the above, the application of feature selection techniques in this study promises several advantages in predicting software vulnerabilities. To obtain the most crucial features for software vulnerability prediction, seven machine learning techniques have been employed: *Pearson Correlation*, *Support Vector Machine*, *Lasso*, *Chi-Squared*, *Recursive Feature Elimination (RFE) with Logistic Regression*, *Recursive Feature Elimination (RFE) with Random Forest*, and *Variance Threshold*.

The rationale behind utilizing diverse techniques is to maximize the performance of machine learning models. Additionally, these specific techniques were selected after considering various factors and conditions, such as the nature of the problem, the size and diversity of the data, and the number of samples in each class.

Subsequent to the application of these feature selection techniques and the identification of the most important features using each method, the features that consistently emerge as the most critical when compared to other features across different methods are retained for further analysis.

C. DATA BALANCING AND NORMALIZATION

Addressing the issue of class imbalance in the dataset is crucial as it is a common challenge in machine learning with various implications for model performance. When one class contains significantly fewer samples than another class, the model may be biased towards the majority class, leading to less accurate results for the minority class. This class imbalance problem can impact machine learning models in two main ways:

- *Increasing the Probability of Overfitting*: If one class has significantly fewer samples, the model might become overly sensitive to that class, resulting in overfitting. This can reduce the model's accuracy when applied to new, unseen data.
- *Decreasing Accuracy*: When a class has very few samples, the model may struggle to correctly identify instances of that class, leading to reduced accuracy for that minority class.

To address the class imbalance problem in this study, the *Smote-Tomek* approach has been utilized. The *Smote-Tomek* approach is a data preprocessing technique designed to tackle class imbalance issues in categorical data. It works by employing two algorithms, *Smote* and *Tomek*, to rebalance the class distribution.

Let's briefly describe each of these two algorithms: (1) *Smote* Algorithm: *Smote* stands for Synthetic Minority Over-sampling Technique. In synthetic data generation, this algorithm uses a framework that interpolates between a minority class sample and its k-nearest neighbors. In our case,

we utilized the default parameter of $k_neighbors = 5$, which means that for each minority instance, five of the closest neighbors are used to generate the synthetic points. The *sampling_strategy* parameter was set so that the distribution of moments reaching similar frequencies for the minority class to the majority class is even. While *Smote* is beneficial for increasing minority samples, it can also create borderline instances, or noisy instances if synthetic points are generated around the overlapping area between the classes. Therefore, it is important to ensure the right parameters are selected, to avoid amplifying noise in the data, and (2) *Tomek* Algorithm: The *Tomek* algorithm highlights instances where *Tomek* links are formed, as pairs of instances from different classes (one from the minority and one from the majority class) are each other's nearest neighbor. We consider these pairs borderline cases, making the class boundary less clear. For the sake of cleaning the decision boundary, we can typically remove the majority class instance from each pair.

The *SMOTE-Tomek* approach employed in this study begins with the *SMOTE* algorithm, which generates synthetic samples from neighboring instances of the minority class to increase its size. This step balances the dataset and reduces the bias introduced by the dominance of the majority class. Subsequently, the *Tomek* link method is applied to identify and eliminate borderline, overlapping, or noisy samples, thereby reducing uncertainty and producing a cleaner and more reliable dataset. Noise reduction also helps to establish clearer decision boundaries and leads to more stable classifiers, ultimately improving classification robustness. The complementary nature of *SMOTE* and *Tomek* is evident: synthetic samples enrich the representation of the minority class, while the removal of noisy and ambiguous samples enhances prediction accuracy and supports stronger generalization.

We chose the *SMOTE-Tomek* technique because the *Tomek* link method removes borderline and noisy samples, ensuring that the dataset is both balanced and of higher quality. By combining data balancing with noise and redundancy reduction, the overall performance of vulnerability prediction models is substantially improved. Furthermore, we recognize that contemporary hybrid techniques can further enhance both data balancing and quality. For instance, study [52] provides evidence that integrating multiple algorithms can simultaneously improve data balance, manage noise, and optimize model performance.

Before applying the *Smote-Tomek* technique, the training dataset contained 21,524 non-vulnerable functions and 235 vulnerable functions. After employing *Smote-Tomek*, the number of vulnerable and non-vulnerable functions reached 21,474 samples. The population and distribution of the data before and after *Smote-Tomek* are shown in Figure 3.

Additionally, as illustrated in Table 2, the metrics in the dataset exhibit different value ranges, which could adversely affect machine learning models. To mitigate this issue, the *Min-Max Normalization* method has been employed to standardize the data. Data normalization is crucial to bring

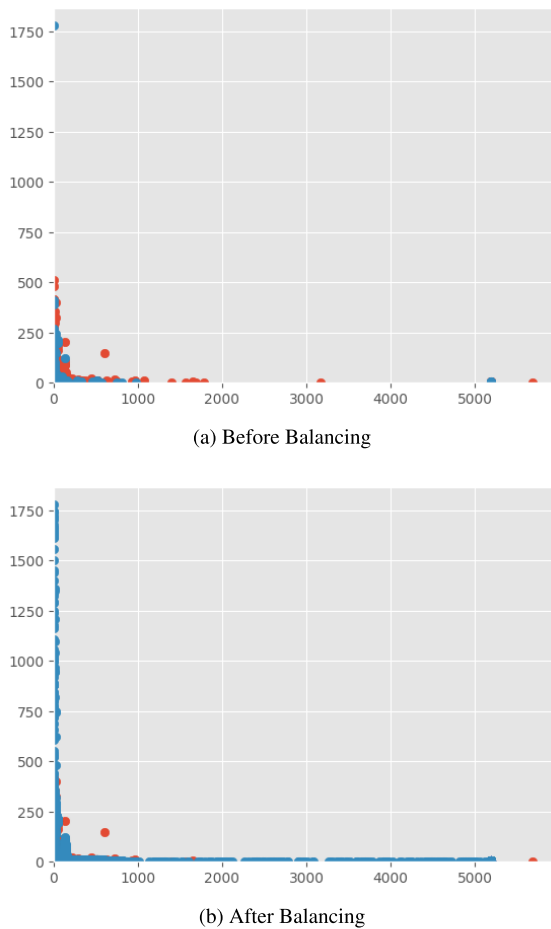


FIGURE 3. Population and dispersion of data before and after Smote-Tomek application.

TABLE 2. Source code metrics ranges.

Metric	Range
M1	1-567
M2	0-37
M3	1-10
M4	0-18
M5	0-5687
M6	0-1779
M7	0-126
M8	0-23
M9	0-393
M10	0-98
M11	0-744
M12	0-400
M13	0-261
M14	0-98

feature values to a similar scale or range, allowing machine learning algorithms to work effectively and efficiently. *Min-Max Normalization* scales values to the range between 0 and 1 using the following formula:

$$\text{NormalizedValue} = (\text{Value} - \text{min}) / (\text{max} - \text{min}) \quad (1)$$

D. CLASSIFICATION

Classification is a fundamental supervised machine learning technique used to predict outcomes based on data. This

research presents an innovative approach to predict software vulnerabilities and detect their types using classification methods, with a focus on enhancing classification accuracy through an ensemble of classifiers. To evaluate the performance of the proposed models, we employed the *Repeated K-Fold Cross-Validation* technique. In this procedure, the dataset is divided into k equally sized folds ($k = 10$ in our implementation). For each iteration, the model is trained on $k - 1$ folds (90% of the data) and tested on the remaining fold (10%). This process is repeated $n_repeats = 3$ times with different random partitions, and the results are averaged to obtain a more robust estimate of model performance. Thus, in each repetition, the training set is constructed from 90% of the data, while the test set consists of the remaining 10%. Since the validation process is inherently embedded in the cross-validation framework, this approach eliminates the need for a separate static validation set and reduces the risk of overfitting. The use of *Repeated K-Fold* ensures that every data point is used multiple times for both training and validation across different iterations, thereby providing a reliable and unbiased performance estimate.

In this study, various machine learning models have been applied, including *logistic regression*, *random forest*, *decision tree*, *XGBoost*, *K-nearest neighbors (KNN)*, *Adaboost*, as well as ensemble methods like the Voting Classifier (comprising RF, KNN, XGB, DT algorithms) and Stacking Classifier (incorporating RF, KNN, XGB, DT algorithms). These diverse models have been leveraged to predict software vulnerabilities and identify their types.

Considering that these models come with a multitude of parameters, *Grid Search* is employed to identify optimal parameter values for machine learning models. By fine-tuning a model's hyperparameters, Grid Search significantly enhances the model's performance when applied to new and unseen data. This meticulous parameter optimization process is pivotal in achieving top-notch results in vulnerability prediction and type detection.

IV. EXPERIMENTAL RESULTS

This section delves into the experimental design and presents the results of various tests conducted to ascertain the pivotal metrics for predicting software vulnerability and discerning vulnerability types. Furthermore, it outlines the outcomes of vulnerability prediction and type detection using these identified crucial metrics. The dataset employed encompasses 15 metrics (see Table 1) extracted from diverse projects' source code, with their specifications. Additionally, Table 3 provides insights into the project specifications. To address data imbalance, *Smote-Tomek* is applied, and *Min-Max Normalization* is utilized to confine the metrics values within a defined range. Subsequently, machine learning models are trained on these balanced datasets and rigorously evaluated based on key metrics including *Accuracy*, *Precision*, *Recall*, and *F1-score*. In total, nine machine learning models were applied to the dataset to optimize vulnerability prediction. To evaluate model performance and prevent overfitting, the

TABLE 3. The hyperparameters of all the machine learning techniques used in this paper.

Machine Learning Technique	Hyperparameter
Gradient Boosting	n_splits=10 n_repeats=3
eXtreme Gradient Boosting	n_estimators= 50 max_depth= 5 gamma= 2 colsample_bytree= 0.8
Logistic Regression	C=10 penalty='l2'
AdaBoost	n_estimators= 200 learning_rate=0.5
K-Nearest Neighbor	max_depth=12 max_leaf_nodes=100 criterion='gini'
Random Forest	n_estimators=100 max_features='sqrt' criterion='gini'
Decision Tree	n_neighbors=3 weights='uniform'
Voting Classifier	estimators = (Random Forest), (K-Nearest Neighbor), (eXtreme Gradient-Boosting), (Decision Tree) voting='soft'
Stacking Classifier	estimators = (K-Nearest Neighbor), (eXtreme GradientBoosting), (Decision Tree) final_estimator=rf

Repeated K-Fold Cross-Validation approach was adopted. Various performance evaluation metrics were employed to assess the models' efficacy. All experiments were conducted using *Python* and its libraries within the *Google Colab* environment.

Table 3 presents details regarding the hyperparameters for all the machine learning techniques employed in this study. Notably, these hyperparameters were determined utilizing the *Grid Search* method.

A. DATASET

Our dataset comprises functions extracted from 10 real projects, publicly accessible projects, as detailed in Table 4. Among these functions, a total of 275 are categorized as vulnerable, as indicated by the "Detection Result" column in the dataset. In this column, a value of 1 signifies vulnerability, while 0 denotes no vulnerability. For the functions identified as vulnerable, the "Vulnerability Type" columns are labeled based on the CVE and NVD databases, covering eight distinct types of vulnerabilities: *DoS*, *Buffer Overflow*, *Memory Corruption*, *SQL Injection*, *Code Execution*, *Bypass* (ASLR Protection, Config, and Sanity Check), *Gain Sensitive Information*, and *Gain Privilege*. It is worth noting that some vulnerability types with a common root have been grouped together to enhance the dataset's accuracy by increasing the number of records for each type.

To achieve one of the primary objectives of this study, which is identifying critical source code metrics for detecting and predicting vulnerability types, we subdivided the main dataset into eight smaller datasets, each corresponding to a specific vulnerability type. This subdivision was motivated

TABLE 4. Information related to under-test programs.

Project	Vulnerable Functions	CVE	Program Functions	LOC
FreeType v2.3.9	74	48	1,859	126,000
LibTIFF v4.0.6	20	12	1,324	118,000
libxslt v1.1.28	5	3	684	47,000
OpenSSL v1.0.1t	42	17	6,674	360,000
SQLite v3.8.2	10	7	4,639	144,000
Binutils v2.28	84	37	22,314	1,617,000
FFmpeg v3.4.1, v3.4.2, v4.0.1, v4.1	12	12	21,394	1,015,000
ImageMagick v7.0.6, v7.0.8	14	17	6,008	361,000
OpenSC v1.8.2, v0.19.0	8	10	4,360	165,000
Rdesktop v1.8.2	6	6	1,050	31,000
Total	275	169	70,306	3,984,000

by the need to address potential class imbalance issues in the dataset, which could lead machine learning models to bias towards the majority class. To rectify class imbalance, the *SmoteTomek* approach was employed, as elaborated in subsequent sections.

In summary, the dataset encompasses 21,759 functions from 10 projects (*SQLite*, *FreeType*, *LibTiff*, *Libxslt*, *Binutils*, *FFmpeg*, *ImageMagick*, *OpenSC*, and *rdesktop*), with 275 of these functions identified as vulnerable. Six of the projects were used for both training and testing, while the remaining four were solely utilized for testing. To fulfill the objective of this study, which includes identifying vulnerability types, eight smaller datasets were constructed to represent each vulnerability type.

B. ANALYSIS

Our initial focus is on conducting a feature importance analysis, a critical step in comprehending which source code metrics play a pivotal role in predicting vulnerabilities, particularly for identifying the types of vulnerabilities. This analysis not only aids in enhancing the understanding of which structural aspects of the code are closely associated with vulnerabilities but also contributes to the development of more secure coding practices.

As previously detailed, we employed seven machine learning techniques, namely *Pearson Correlation*, *Support Vector Machine*, *Lasso*, *Chi-Squared*, *Recursive Feature Elimination (RFE) with Logistic Regression*, *Recursive Feature Elimination (RFE) with Random Forest*, and *Variance Threshold*, to extract the most significant features and metrics in this context. After applying these methods and identifying

TABLE 5. Analysis of the most important features of the code in detecting the type of vulnerability.

All Types	DoS	Bypass sth	Code Execution	Gain privilege	Gain sensitive info	Memory corruption	Buffer overflow	SQL injection
M5	M1	M5	M5	M1	M1	M1	M5	M1
M6	M5	M6	M1	M11	M11	M6	M1	M12
M1	M6	M1	M6	M5	M5	M5	M6	M6
M9	M7	M9	M7	M6	M6	M11	M12	M9
M11	M9	M12	M9	M7	M7	M12	M9	M2

the most important features using each one, we selected the top five features that exhibited the highest importance among all features according to each method. These five crucial characteristics, as determined for both vulnerability prediction and vulnerability type detection, are presented in Table 5. As seen, metrics M5, M6, M1, M9, and M11 have emerged as the most crucial factors in forecasting vulnerabilities.

Criterion M5 is closely linked to the number of variables within the argument of the Caller function. A higher count of variables in the arguments of the Caller function correlates with an elevated vulnerability likelihood. A larger number of arguments in the Caller function increases the probability of incorrect input entry, potentially leading to security breaches. Moreover, a surplus of arguments contributes to code complexity. To mitigate security vulnerabilities in the Caller function, it is advisable to minimize the number of arguments, retaining only those essential for the function's operation. Additionally, secure input validation techniques should be employed to prevent the entry of invalid or malicious data into the Caller function.

Criterion M6 assesses the prevalence of arithmetic pointers in the program code. It quantifies how frequently arithmetic pointers are utilized in the code. These pointers increment or decrement the pointer's value from the current address, rather than pointing to a specific address. Improper usage of arithmetic pointers can introduce security vulnerabilities. Therefore, programs with fewer arithmetic pointers are less susceptible to vulnerabilities.

Criterion M1 pertains to the Cyclomatic complexity of the program, which measures code complexity. It quantifies the number of independent paths within a program, effectively counting the logical branches within the code. A lower Cyclomatic complexity indicates a reduced vulnerability likelihood and enhanced maintainability and development capabilities.

Criterion M9 evaluates the number of Nested Control Structure Terms (CSTs) within the code. Nested CSTs arise when one control structure is nested within another, such as a loop within a condition. An excessive number of nested CSTs augments code complexity, potentially fostering software vulnerabilities. To preempt such vulnerabilities, it is advisable to minimize the occurrence of nested CSTs in the code. Techniques to reduce code complexity, such as employing smaller and simpler functions, substituting loops with recursive functions, and utilizing appropriate synchronization methods, can be employed.

Criterion M11 characterizes the maximum dependency control between code control structures. Excessive dependencies between control structures increase code complexity and the vulnerability probability.

Regarding the diagnosis of Denial of Service (DoS) type vulnerabilities, metrics M1, M5, M6, M7, and M9 have been identified as the most influential metrics, with criterion M7 being an additional factor in predicting vulnerabilities. Criterion M7 pertains to the number of variables related to arithmetic pointers, highlighting its significance in the context of DoS vulnerabilities.

Summarily, the results delineated in Table 5 underscore that metrics M1 and M6 are consistently the most critical across both vulnerability prediction and the diagnosis of the eight existing vulnerability types. Thus, the number of variables within the Caller function's argument and the prevalence of arithmetic pointers emerge as pivotal determinants in software vulnerability.

As outlined in the Proposed Model Section, after ascertaining the five critical features for both vulnerability prediction and the diagnosis of the eight vulnerability types, we applied the *SmoteTomek* and *Min-Max Normalization* techniques to the dataset. Subsequently, various machine learning models were implemented, with the results of nine such models in predicting vulnerabilities based on the five essential features presented in Figure 4.

Figure 4 offers a comprehensive view of the model performance across various metrics, including *Accuracy*, *Precision*, *Recall*, and *F1-Score*. Notably, the *Random Forest* model stands out as the top-performing model across all these metrics. It boasts an *Accuracy* of 0.96, a *Precision* of 0.948, a *Recall* of 0.974, and an *F1-Score* of 0.961. This outcome signifies a significant advancement in vulnerability prediction compared to prior studies. Following the *Random Forest* model, the *Voting Classifier* demonstrated superior performance compared to other machine learning models across all metrics.

Figure 5 provides an overview of the Confusion Matrix for all implemented models, enhancing our understanding of the results. Upon a thorough evaluation and analysis of the results, as depicted in Figure 5, it is evident that the *Random Forest* model delivered the most exceptional performance across all metrics. Examining the results in the Confusion Matrix validates the *Random Forest* model's efficacy, with a True Positive (TP) rate of 0.93 and a True Negative (TN) rate of 0.97, indicating its strong performance in detecting both vulnerabilities and non-vulnerabilities. Furthermore, the



FIGURE 4. Comparing the performance of 9 machine learning algorithms to detect the vulnerable functions.

Random Forest model boasts a very low False Negative (FN) rate of 0.07 and a False Positive (FP) rate of 0.03, demonstrating minimal prediction errors.

Table 6 presents an overview of the number of vulnerable functions identified within each of the ten projects considered in this study, and it compares the performance of our proposed model with that of previous works. It is evident that our proposed model outperforms prior efforts in all projects. For instance, in the *Binutils* project, where there are 84 vulnerable functions, our model correctly identified 79 of them. In contrast, the best results from previous works for the *Binutils* project only identified 72 vulnerable functions. Across all the projects assessed, our prediction model successfully identified 261 vulnerable functions out of 275 total functions, surpassing the 244 vulnerable functions identified in our previous work [42]. As noted in the Figure 4 explanations, our proposed model achieved an accuracy of 0.96 in predicting vulnerable functions across all projects under scrutiny. Importantly, our learning model had no prior knowledge of the vulnerable functions in four projects (FFmpeg, ImageMagick, OpenSC, and rdesktop). Nevertheless, Table 6 demonstrates a significant improvement in the detection outcomes of vulnerable functions using our proposed model across all projects. Notably, for the *FFmpeg*, *ImageMagick*, *OpenSC*, *rdesktop*, and *SQLite* projects, our model effectively identified all existing vulnerabilities.

Figure 5 showcases the results of applying nine machine learning models to eight datasets related to the type of vulnerability. It is evident from the figure that the Random Forest model outperformed all other models in detecting all eight types of vulnerabilities. Consequently, we can assert that we have arrived at a single and optimal model for vulnerability

prediction and the identification of vulnerability types. This achievement was made possible by combining approaches such as feature selection, data balancing and normalization, and fine-tuning model parameters. For instance, in detecting DoS-type vulnerabilities, the Random Forest model achieved an Accuracy of 0.972, Precision of 0.966, Recall of 0.978, and F1-Score of 0.972. In our previous work [42], the Accuracy for detecting DoS-type vulnerabilities was only 0.89.

C. DISCUSSION OF CONTRIBUTIONS AND COMPARISON WITH RECENT ADVANCES

1) INTERPRETATION OF RESULTS AND CODE-LEVEL INSIGHTS

The consistently strong performance of the Random Forest model in our experiments can be explained by its intrinsic algorithmic advantages. By combining the outputs of many decision trees, Random Forest lowers variance, thereby reducing the likelihood of overfitting and supporting more stable generalization. Unlike single learners, such as logistic regression or shallow decision trees, it manages noisy software metrics and correlations among features more effectively. This capacity likely explains why it surpassed boosting-based approaches and neural network architectures across both precision and recall measures.

Equally noteworthy are the code-level features that emerged as critical indicators in our analysis. Metrics such as Cyclomatic Complexity (M1), pointer arithmetic usage (M6), nested control structures (M9), and control dependencies (M11) reveal structural aspects of source code that are closely tied to security risks. For instance, excessive complexity or deep nesting often conceals logical flaws, while intensive reliance on pointer



FIGURE 5. Comparing the confusion matrices of nine machine learning algorithms to detect the vulnerable functions.

TABLE 6. Comparing the output of the proposed model and previous work in terms of detecting the number of vulnerable functions in 10 real projects.

Project	Number of vulnera-ble functions	Number of vulnerable functions detected in the previous studies	Number of vulnerable functions detected in our previous work [43]	Number of vulnerable functions detected in the proposed model
FreeType	74	56 [44]	64 [43]	67
LibTIFF	20	16 [44] 17 [60]	17 [43]	19
libxslt	5	5 [44]	5 [43]	5
OpenSSL	42	37 [44]	39 [43]	41
SQLite	10	6 [44]	7 [43]	10
Binutils	84	55 [44]	72 [43]	79
FFmpeg	12	4[18], 7[21], 11[45], 5[46], 6[47], 8[48], 11[49]	12 [43]	12
ImageMagick	14	1[18], 5[21], 5[45], 5[46], 11[47], 10[48], 13[49]	14 [43]	14
OpenSC	8	1[21], 1[45], 5[46], 4[47], 5[48], 8[49]	8 [43]	8
rdesktop	6	2[21], 2[45], 3[46], 3[47], 4[48], 6[49]	6 [43]	6

arithmetic increases the risk of buffer overflows or memory corruption. These results provide actionable insights for developers, suggesting that simplifying control flows, limiting pointer operations, and applying conditionals



FIGURE 6. Comparing the performance of 9 machine learning algorithms to detect eight different attack types.

with discipline can significantly reduce vulnerability exposure.

When interpreted in the broader cybersecurity context, these findings gain additional significance. Prior comparative research on Neural Networks and Support Vector Machines in security tasks [57] has shown that no single method dominates all applications, with performance strongly shaped by the nature of the input data. Our observations align with this view: while neural networks may excel in tasks such as anomaly detection or intrusion monitoring, Random Forest demonstrated superior reliability for vulnerability prediction grounded in code metrics. This underscores the need for algorithm selection to remain task-specific rather than universally prescriptive.

In conclusion, Random Forest not only achieves high predictive accuracy but also bridges statistical performance with interpretable, actionable recommendations for secure software design. This combination of robustness and clarity positions it as a particularly valuable approach in advancing both the theoretical understanding and the practical management of software vulnerabilities.

2) COMPARISON WITH LARGE LANGUAGE MODEL APPROACHES

Our contribution to the field of software vulnerability prediction is distinctive in that we demonstrate the effectiveness of machine learning techniques combined with substantial source code metrics. While recent studies have explored the use of large language models (LLMs) for vulnerability detection—leveraging their ability to interpret code semantics similarly to natural language—our findings show that a targeted approach using well-defined software metrics is both valid and capable of producing strong results. The work in [53] highlights the promise of LLMs for analyzing code and identifying vulnerabilities. However, these models often operate as “black boxes,” offering limited interpretability and requiring significant computational resources for training and inference. In contrast, our approach provides a transparent and efficient framework that consumes fewer resources. The model delivers precise insights into structural and logical characteristics—such as Cyclomatic complexity, the number of arithmetic pointers, and the depth of nested control structures—that frequently underlie coding and design vulnerabilities. This interpretability empowers developers to directly address the root causes of weaknesses in their code, rather than relying on opaque black-box predictions.

D. LIMITATIONS AND SOURCES OF UNCERTAINTY

Nonetheless, the suggested approach is still subject to a few limitations, which should be weighed against the advantages of the approach.

Firstly, building Code Property Graphs is based on static analysis and, as such, can introduce some inaccuracies. For example, conservative approximations can lead to false, positive edges in control flow graphs, especially when indirect function calls or unresolved dynamic dispatch are

involved. Likewise, the generation of an abstract syntax tree can be disturbed by the presence of macros, preprocessor directives, and compiler-specific extensions that are very common in C/C++ projects. All these elements may cause some deterioration of the graphs.

Nevertheless, it is worth recalling that our model does not make predictions at the level of nodes or edges, but works rather on aggregated structural metrics. This aggregation greatly reduces the negative effects of localized mistakes in graph construction and, at the same time, enables the learning algorithm to identify broader structural patterns that are less different across projects.

Besides, there is a limitation related to the applicability of the approach. The current work is carried out on C/C++ codebases, which are chosen due to the fact that these languages are extensively used in low-level systems software, and, hence, are very vulnerable to memory-related issues. Moreover, the Code Property Graph extraction relies on the JOERN framework, which can also affect the availability of metrics and fidelity of graphs.

A second limitation is related to the scope of applicability. This research concentrates on C/C++ codebases since these languages are widely used for low-level systems software and are known for memory-related vulnerabilities. Furthermore, the Joern framework, which is required for CPG extraction, may determine the metric availability and graph fidelity. Although these decisions are well, argued with respect to the vulnerability classes under consideration, they may reduce the ease of application to other programming languages without adaptation. From the standpoint of practical deployment, the computational complexity of the proposed system is not very high. Construction of graphs and extraction of metrics unfold almost linearly with the size of code; besides that, the use of Random Forest for model inference results in a negligible overhead after the training phase. Hence, the method can be conveniently carried out offline and be part of a CI/CD pipeline, especially at the pre-merge or scheduled security assessment stages where immediate feedback is not required. To sum up, Random Forest not only predicts accurately but also effectively combines statistical performance with interpretable, actionable recommendations for secure software design. The proposed framework, thus, by virtue of its robustness, transparency, and practical feasibility, can be considered a valuable addition to both the theoretical knowledge and real-world handling of software vulnerabilities.

E. PRACTICAL IMPLICATIONS AND SECURITY GUIDELINES

Through our empirical analysis of source code metrics, we have identified critical threshold points that developers need to keep in mind so that their code doesn't have loopholes that others can misuse. We have drawn up some technical guidelines based on how metrics and patterns of vulnerabilities are correlated: Combinations resulting in high value of Cyclomatic Complexity (M1) and large number of control structures nested very deeply (M9) greatly increase

the program state space. Looking from the security point of view, the complexity like that limits the extent to which the software can be formally verified, and also it becomes more difficult for developers to understand the code. Hence, they become more susceptible to hidden execution paths and logic vulnerabilities that could be very subtle. Thus, the developers are advised to control flow simplification by means of refactoring techniques, the use of design patterns like the Strategy Pattern, or Table-Driven Methods. These style changes lead to reduced control, flow depth, enhanced clarity of execution, path, and static analysis tools are more effective in auditing the code. The concentration of arithmetic pointer (M6) is one of the main symptoms of memory corruption, which is very likely. In low-level languages, Buffer Overflows and Use-After-Free vulnerabilities often stem directly from incorrect and unchecked pointer usage. We suggest that in cases when M6 gets beyond the safe limit figured out, the developers should put in place very accurate bounds, checking, or even switch to smart pointers/managed memory constructs to wrap up the pointer logic. A large amount of M5 (variables prepared for recalled functions) would indicate a complicated “Information Flow” between the modules. This kind of situation usually results in Insecure Deserialization or Injection attacks if the data is not properly cleaned before being handed over to the next function. Developers are urged to follow the “Strong Typing” and “Interface Segregation” principles to make sure that only the necessary data is shared between the caller and the callee. Maximum control dependency (M11) refers to the degree to which conditionals control the execution of the program. Such situations can be exploited via Timing Attacks or any other side-channel attacks. One way to reduce M11 is to separate sensitive logic from complicated control structures so that execution time and resource usage remain the same, thus preventing information leakage.

F. FUTURE WORK AND AI-DRIVEN DEFENSE

While this study limits itself to C/C++ codebases, because they account for most low, level systems and memory, related vulnerabilities, it is a good idea for future research to extend the proposed framework to other programming languages like Python and Java. Multi, language coding environments would require code property graph extraction and metric computation to be adjusted. This would open up the method for more applications and make it easier to predict vulnerabilities in modern software ecosystems, such as web and enterprise applications.

Besides static analysis, it is becoming a must to combine predictive models with autonomous security systems nowadays. AI, powered cybersecurity research has demonstrated a move from offline vulnerability identification to realtime threat detection and proactive defense mechanisms [58]. Therefore, future work may find adaptive security frameworks where ML models, using source code metrics, not only predict potential defects but also, dynamically initiating

preventive actions like firewall rules updates, runtime monitoring, or sandboxing, on detecting anomalous behavior.

The incorporation of multi, language code metrics with AI, based early detection and preventive measures is hence anticipated to be a major aspect of future cyber defense systems.

V. CONCLUSION

Processing code-derived metrics with machine learning algorithms offers a robust solution for detecting vulnerabilities and mitigating associated risks, such as unauthorized system intrusions. By meticulously analyzing these metrics and the influential factors governing software vulnerability, it becomes possible to proactively prevent vulnerabilities through secure coding practices. In this study, we present an efficient machine learning-based approach for predicting software vulnerabilities and detecting their types. Our methodology encompasses a range of feature selection techniques, including *Pearson Correlation*, *Support Vector Machine*, *Lasso*, *Chi-Squared*, *Recursive Feature Elimination (RFE) with Logistic Regression*, *Recursive Feature Elimination (RFE) with Random Forest*, and *Variance Threshold*. Notably, Cyclomatic complexity and the count of arithmetic pointers within the code emerged as two of the most salient metrics.

Furthermore, our research employs a diverse set of machine learning models, encompassing *Gradient Boosting*, *eXtreme Gradient Boosting*, *Random Forest*, *Logistic Regression*, *Decision Tree*, *AdaBoost*, *Voting Classifier*, and *Stacking Classifier*. We measure their performance using the five most crucial metrics extracted through distinct feature selection methodologies. To address class imbalance, we implement the *SmoteTomek* technique. Our experimental results affirm the effectiveness of amalgamating these diverse approaches, combining feature selection, *SmoteTomek* utilization, and parameter fine-tuning through Grid-Search. Notably, the *Random Forest* model emerges as the most proficient, boasting an impressive Accuracy of 0.96, Precision of 0.948, Recall of 0.974, and F1-score of 0.961.

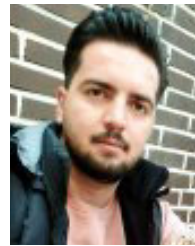
This research stands to substantially enhance software system vulnerability management, serving as an invaluable tool for software developers. It equips developers with insights into secure coding practices and enhances their understanding of software security. Looking ahead, future iterations of this study should consider expanding beyond the confines of C and C++ languages for vulnerability prediction. Additionally, exploring the automation of source code metrics extraction through deep learning models represents an exciting avenue for further research.

REFERENCES

- [1] H. Shahriar and M. Zulkernine, “Mitigating program security vulnerabilities: Approaches and challenges,” *ACM Comput. Surv.*, vol. 44, no. 3, pp. 1–46, Jun. 2012.
- [2] Y. Shin and L. Williams, “Can traditional fault prediction models be used for vulnerability prediction?” *Empirical Softw. Eng.*, vol. 18, no. 1, pp. 25–59, Feb. 2013.

- [3] H. Hanif, M. H. N. Md Nasir, M. F. Ab Razak, A. Firdaus, and N. B. Anuar, "The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches," *J. Netw. Comput. Appl.*, vol. 179, Apr. 2021, Art. no. 103009.
- [4] B. Chernis and R. Verma, "Machine learning methods for software vulnerability detection," in *Proc. 4th ACM Int. Workshop Secur. Privacy Anal.*, Mar. 2018, pp. 31–39.
- [5] F. Yamaguchi, M. Lottmann, and K. Rieck, "Generalized vulnerability extrapolation using abstract syntax trees," in *Proc. 28th Annu. Comput. Secur. Appl. Conf.*, Dec. 2012, pp. 359–368.
- [6] Z. Durumeric, F. Li, J. Kasten, J. Amann, J. Beekman, M. Payer, N. Weaver, D. Adrian, V. Paxson, M. Bailey, and J. A. Halderman, "The matter of heartbleed," in *Proc. Conf. Internet Meas. Conf.*, Nov. 2014, pp. 475–488.
- [7] S. Peldszus, J. Bürger, T. Kehler, and J. Jürjens, "Ontology-driven evolution of software security," *Data Knowl. Eng.*, vol. 134, Jul. 2021, Art. no. 101907.
- [8] J. Bürger, T. Kehler, and J. Jürjens, "Ontology evolution in the context of model-based secure software engineering," in *Proc. Int. Conf. Res. Challenges Inf. Sci. Cham, Switzerland: Springer*, Jun. 2020, pp. 437–454.
- [9] J. Jürjens and P. Shabalin, "Tools for secure systems development with UML," *Int. J. Softw. Tools Technol. Transf.*, vol. 9, nos. 5–6, pp. 527–544, Oct. 2007.
- [10] H. Guan, X. Wang, and H. Yang, "A framework for security driven software evolution," in *Proc. 20th Int. Conf. Autom. Comput.*, Sep. 2014, pp. 194–199.
- [11] Z. Yu, C. Theisen, L. Williams, and T. Menzies, "Improving vulnerability inspection efficiency using active learning," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2401–2420, Nov. 2021.
- [12] F. Yamaguchi and K. Rieck, "Vulnerability extrapolation: Assisted discovery of vulnerabilities using machine learning," in *Proc. 5th USENIX Workshop Offensive Technologies (WOOT)*, 2011, pp. 118–127.
- [13] Y. Pang, X. Xue, and A. S. Namin, "Predicting vulnerable software components through N-gram analysis and statistical feature selection," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2015, pp. 543–548.
- [14] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2018, pp. 757–762.
- [15] B. Arkin, S. Stender, and G. McGraw, "Software penetration testing," *IEEE Secur. Privacy*, vol. 3, no. 1, pp. 84–87, Feb. 2005.
- [16] T. Zimmermann, N. Nagappan, and L. Williams, "Searching for a needle in a haystack: Predicting security vulnerabilities for windows vista," in *Proc. 3rd Int. Conf. Softw. Test., Verification Validation*, Apr. 2010, pp. 421–428.
- [17] S. Kim, S. Woo, H. Lee, and H. Oh, "VUDDY: A scalable approach for vulnerable code clone discovery," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 595–614.
- [18] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, "Vulpecker: An automated vulnerability detection system based on code similarity analysis," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 201–213.
- [19] R. Scandariato, J. Walden, A. Hovsepian, and W. Joosen, "Predicting vulnerable software components via text mining," *IEEE Trans. Softw. Eng.*, vol. 40, no. 10, pp. 993–1006, Oct. 2014.
- [20] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," 2018, *arXiv:1801.01681*.
- [21] L. Wartschinski, Y. Noller, T. Vogel, T. Kehler, and L. Grunske, "VUDENC: Vulnerability detection with deep learning on a natural code-base for Python," *Inf. Softw. Technol.*, vol. 144, Apr. 2022, Art. no. 106809.
- [22] B. Baloglu, "How to find and fix software vulnerabilities with coverity static analysis," in *Proc. IEEE Cybersecurity Develop. (SecDev)*, Nov. 2016, p. 153.
- [23] V. V. Desai and V. J. Jariwala, "Comprehensive empirical study of static code analysis tools for C language," *Int. J. Intell. Syst. Appl. Eng.*, vol. 10, no. 4, pp. 695–700, 2022.
- [24] D. Vitek, "Auditing code for security vulnerabilities with CodeSonar," in *Proc. IEEE Cybersecurity Develop. (SecDev)*, Nov. 2016, p. 154.
- [25] N. Ayewah, W. Pugh, J. D. Morgenthaler, J. Penix, and Y. Zhou, "Using FindBugs on production software," in *Proc. Companion 22nd ACM SIGPLAN Conf. Object-Oriented Program. Syst. Appl. companion*, Oct. 2007, pp. 805–806.
- [26] Y. Wang, G. Xu, X. Liu, W. Mao, C. Si, W. Pedrycz, and W. Wang, "Identifying vulnerabilities of SSL/TLS certificate verification in Android apps with static and dynamic analysis," *J. Syst. Softw.*, vol. 167, Sep. 2020, Art. no. 110609.
- [27] S. Rahalkar, "Testing mobile apps and APIs with burp suite," in *A Complete Guide to Burp Suite: Learn to Detect Application Vulnerabilities*. Berkeley, CA, USA: Apress, 2021, pp. 147–164.
- [28] L. J. Moukahal, M. Zulkermine, and M. Soukup, "Vulnerability-oriented fuzz testing for connected autonomous vehicle systems," *IEEE Trans. Rel.*, vol. 70, no. 4, pp. 1422–1437, Dec. 2021.
- [29] H. Homaei and H. R. Shahriari, "Athena: A framework to automatically generate security test Oracle via extracting policies from source code and intended software behaviour," *Inf. Softw. Technol.*, vol. 107, pp. 112–124, Mar. 2019.
- [30] J. A. Harer, L. Y. Kim, R. L. Russell, O. Ozdemir, L. R. Kosta, A. Rangamani, L. H. Hamilton, G. I. Centeno, J. R. Key, P. M. Ellingwood, E. Antelman, A. Mackay, M. W. McConley, J. M. Oppen, P. Chin, and T. Lazovich, "Automated software vulnerability detection with machine learning," 2018, *arXiv:1803.04497*.
- [31] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.
- [32] D. Hin, A. Kan, H. Chen, and M. A. Babar, "LineVD: Statement-level vulnerability detection using graph neural networks," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 596–607.
- [33] Y. Shin and L. Williams, "An empirical model to predict security vulnerabilities using code complexity metrics," in *Proc. 2nd ACM-IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Oct. 2008, pp. 315–317.
- [34] K. Z. Sultana, A. Deo, and B. J. Williams, "A preliminary study examining relationships between nano-patterns and software security vulnerabilities," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, vol. 1, Jun. 2016, pp. 257–262.
- [35] G. Lin, J. Zhang, W. Luo, L. Pan, and Y. Xiang, "POSTER: Vulnerability discovery with function representation learning from unlabeled projects," in *Proc. ACM SIGSAC Conf. Comput. Communications Secur.*, Oct. 2017, pp. 2539–2541.
- [36] X. Ban, S. Liu, C. Chen, and C. Chua, "A performance evaluation of deep-learned features for software vulnerability detection," *Concurrency Comput., Pract. Exper.*, vol. 31, no. 19, Oct. 2019, Art. no. e5103.
- [37] Q. Meng, S. Wen, B. Zhang, and C. Tang, "Automatically discover vulnerability through similar functions," in *Proc. Prog. Electromagn. Res. Symp. (PIERS)*, Aug. 2016, pp. 3657–3661.
- [38] G. Jie, K. Xiao-Hui, and L. Qiang, "Survey on software vulnerability analysis method based on machine learning," in *Proc. IEEE 1st Int. Conf. Data Sci. Cyberspace (DSC)*, Jun. 2016, pp. 642–647.
- [39] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul. 2022.
- [40] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, and Y. Acar, "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 426–437.
- [41] C. Chen, R. Vitenberg, and H.-A. Jacobsen, "Building fault-tolerant overlays with low node degrees for topic-based Publish/Subscribe," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 5, pp. 3011–3023, Sep. 2022.
- [42] P. Pakshad, A. Shameli-Sendi, and B. K. E. Abbasi, "A security vulnerability predictor based on source code metrics," *J. Comput. Virol. Hacking Techn.*, vol. 19, no. 4, pp. 615–633, Feb. 2023.
- [43] X. Du, B. Chen, Y. Li, J. Guo, Y. Zhou, Y. Liu, and Y. Jiang, "LEOPARD: Identifying vulnerable code for vulnerability assessment through program metrics," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, May 2019, pp. 60–71.
- [44] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "VulDeePecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Trans. Depend. Secure Comput.*, vol. 18, no. 5, pp. 2224–2236, Oct. 2021.
- [45] G. Lin, J. Zhang, W. Luo, L. Pan, O. De Vel, P. Montague, and Y. Xiang, "Software vulnerability discovery via learning multi-domain knowledge bases," *IEEE Trans. Depend. Secure Comput.*, vol. 18, no. 5, pp. 2469–2485, Sep. 2021.
- [46] M. Pradel and K. Sen, "DeepBugs: A learning approach to name-based bug detection," *Proc. ACM Program. Lang.*, vol. 2, pp. 1–25, Oct. 2018.

- [47] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 1–11.
- [48] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1943–1958, 2021.
- [49] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, "Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities," *IEEE Trans. Softw. Eng.*, vol. 37, no. 6, pp. 772–787, Nov. 2011.
- [50] I. Chowdhury and M. Zulkernine, "Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities," *J. Syst. Archit.*, vol. 57, no. 3, pp. 294–313, Mar. 2011.
- [51] Y. Zhou and A. Sharma, "Automated identification of security issues from commit messages and bug reports," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, Aug. 2017, pp. 914–919.
- [52] M. G. Yaseen, A. H. Ali, and M. Alajjanbi, "An enhanced hybrid genetic-JAYA algorithm for feature selection and SVM parameter optimization in intrusion detection systems: Evaluation on the CICIDS dataset," in *Proc. SHIFRA*, 2025, pp. 98–109.
- [53] A. Ali and M. C. Ghanem, "Beyond detection: Large language models and next-generation cybersecurity," in *Proc. SHIFRA*, 2025, pp. 81–97.
- [54] D. Goodin, "An NSA-derived ransomware worm is shutting down computers worldwide," *Ars Technica*, May 2017.
- [55] A. Mechri, M. A. Ferrag, and M. Debbah, "SecureQwen: Leveraging LLMs for vulnerability detection in Python codebases," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104151.
- [56] L. Hussain, "Fortifying Ai against cyber threats: Advancing resilient systems to combat adversarial attacks," in *Proc. EDRAAK*, 2024, pp. 26–31.
- [57] A. K. Abed, "Utilizing artificial intelligence in cybersecurity: A study of neural networks and support vector machines," *Babylonian J. Netw.*, vol. 2025, pp. 14–24, Feb. 2025.
- [58] M. A. Khalaf and A. Steiti, "Artificial intelligence predictions in cyber security: Analysis and early detection of cyber attacks," *Babylonian J. Mach. Learn.*, vol. 2024, pp. 63–68, May 2024.
- [59] Y. Zhao, G. Lin, and Z. Liao, "Long-range context modeling for software vulnerability detection using an XLNet-based approach," *Sci. Rep.*, vol. 16, no. 1, pp. 1–16, Jan. 2026.



BEHZAD KHALAJI EMAMZADEH ABBASI received the B.Sc. degree from Bu-Ali Sina University, Hamedan, Iran, and the M.Sc. degree in computer engineering from Isfahan University of Technology, Isfahan, Iran. He is currently pursuing the Ph.D. degree with Shahid Beheshti University, Tehran, Iran. His doctoral project is in the field of software code vulnerabilities detection. He has extensive experience in machine learning techniques. His research interests include vulnerability detection, deep learning, and cloud computing.



ALIREZA SHAMELI-SENDI received the B.Sc. and M.Sc. degrees from the Amirkabir University of Technology and the Ph.D. degree in computer engineering from the University of Montreal (École Polytechnique de Montréal), Canada. He is currently an Associate Professor with Shahid Beheshti University. Before joining SBU, he was a Postdoctoral Fellow with Ericsson, Canada; and a Postdoctoral Researcher with ETS and McGill universities in collaboration with Ericsson. His primary research interests include information security, intrusion response systems, and cloud computing. He was a recipient of the Postdoctoral Research Fellowship Award and the Industrial Postdoctoral Fellowship Award from Canada. In addition, he received the Best Researcher Awards at SBU, in 2018 and 2022.

...